Scalable Real-time Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

Ammar Yasir Alharbi

4714241

College of Computer Science and Engineering, Taibah University

MSBDA - Big Data Analysis

Dr. Mohammed Al-Sarem

Abstract

The rapid advancement of artificial intelligence (AI), particularly large language models (LLMs), has enabled the generation of highly fluent and human-like Arabic text. This has introduced significant challenges in distinguishing between human-written and AI-generated content, raising concerns about authorship authenticity and academic integrity.

This project aims to develop a scalable and efficient system for detecting AI-generated Arabic text using big data processing and machine learning techniques. A large dataset consisting of both human-written and AI-generated Arabic abstracts was processed using Apache Spark to support large-scale analysis.

The proposed approach combines stylometric features, such as the number of words with repeated letters and sentence count, with TF-IDF representations to capture both linguistic and statistical patterns. Multiple machine learning models were trained and evaluated, including Logistic Regression, Support Vector Machines (SVM), and XGBoost.

The experimental results show that Logistic Regression achieved the best overall balanced performance, accurately classifying both human-written and AI-generated text with high reliability. While SVM also performed strongly, XGBoost demonstrated a bias toward the AI class, leading to poor generalization for human-written text.

Overall, this study demonstrates that combining big data frameworks with machine learning provides an effective solution for detecting AI-generated Arabic content, with potential applications in real-time text monitoring systems.

Introduction

Recent advancements in artificial intelligence, particularly in natural language processing (NLP), have significantly improved the ability of machines to generate human-like text. Large language models (LLMs), such as GPT and LLaMA, are capable of producing coherent and context-aware content, making it increasingly difficult to distinguish between human-written and machine-generated text.

This rapid growth in AI-generated content has raised serious concerns regarding authorship verification, academic integrity, and content reliability. Detecting AI-generated text has therefore become a critical challenge, especially in academic, research, and digital communication domains.

The problem becomes more complex in the context of the Arabic language, which presents unique challenges due to its rich morphology, complex grammatical structures, and diverse dialects. These characteristics make both text generation and detection more difficult compared to other languages.

This project aims to address this challenge by developing a scalable system for detecting AI-generated Arabic text using big data technologies and machine learning techniques. Apache Spark is used as the core framework to enable efficient distributed processing of large-scale datasets.

The proposed approach combines both stylometric features and advanced text representation techniques. Two handcrafted features, namely repeated letter patterns and sentence count, were used to capture writing style differences, while TF-IDF was applied to represent text in a high-dimensional feature space.

Several machine learning models were implemented and evaluated, including Logistic Regression, Support Vector Machines (SVM), and XGBoost. Additionally, a real-time streaming pipeline was developed using Spark Structured Streaming to simulate live deployment scenarios.

The primary objective of this project is to build an accurate and scalable system capable of distinguishing between human-written and AI-generated Arabic text while analyzing the key features that influence classification decisions.

Rekated work

The problem of distinguishing between human-written and AI-generated text has recently attracted significant attention in the field of natural language processing (NLP), particularly with the rapid advancements in large language models (LLMs). These models, including GPT, LLaMA, and other transformer-based architectures, have demonstrated an unprecedented ability to generate highly coherent and contextually relevant text, making authorship detection increasingly challenging.

Most existing research in this domain has focused primarily on English-language datasets. Early approaches relied on traditional machine learning techniques such as Logistic Regression, Support Vector Machines (SVM), and Random Forest classifiers. These methods utilized handcrafted stylometric features, including sentence length, lexical diversity, punctuation usage, and repetition patterns, to capture differences between human and machine-generated writing styles.

More advanced studies have explored deep learning and embedding-based methods, where textual data is transformed into dense vector representations using techniques such as Word2Vec, GloVe, and more recently, transformer-based embeddings like BERT. These approaches have been shown to capture semantic and contextual nuances within text, leading to improved classification performance in many cases.

Despite these developments, research on Arabic text classification, particularly for detecting AI-generated content, remains limited. The Arabic language presents unique challenges due to its rich morphology, complex grammatical structures, varying orthographic forms, and the presence of multiple dialects. These characteristics significantly impact both text generation and detection, making models developed for English less effective when applied directly to Arabic data.

Some studies have begun to explore the use of machine learning techniques for Arabic text classification, focusing on tasks such as sentiment analysis and authorship attribution. However, the specific problem of detecting AI-generated Arabic text is still relatively under-explored, highlighting the need for more targeted approaches.

This project contributes to this emerging area by combining both stylometric and statistical text representation techniques within a scalable big data framework. Specifically, it incorporates handcrafted features such as repeated letter patterns and sentence count alongside TF-IDF representations, enabling the model to capture both structural and frequency-based characteristics of text.

Unlike purely deep learning-based approaches, this study emphasizes the effectiveness of traditional machine learning models when combined with efficient feature engineering and large-scale data processing using Apache Spark. This approach provides a practical and computationally efficient solution for detecting AI-generated Arabic content, while also offering interpretability through feature importance analysis.

Data Description

The dataset used in this project consists of Arabic research abstracts collected from multiple sources, including both human-written and AI-generated content. The primary objective of the dataset is to provide a comprehensive and balanced representation of different writing styles to enable effective classification.

The dataset is obtained from the Hugging Face repository "KFUPM-JRCAI/arabic-generated-abstracts" and includes multiple subsets such as:

1. by_polishing
2. from_title
3. from_title_and_content

Each subset contains several columns, including:

1. Original_abstract (human-written text)

2. AI-generated abstracts produced by different models such as Allam, Jais, LLaMA, and OpenAI

To build the classification dataset, all subsets were merged into a unified Spark DataFrame. The data was then transformed into a binary classification format, where:

1. Label 0 represents human-written abstracts
2. Label 1 represents AI-generated abstracts

The final dataset contains approximately 41,940 text samples, consisting of:

1. 8,388 human-written abstracts
2. 33,552 AI-generated abstracts

Although the dataset is imbalanced, with a significantly higher number of AI-generated samples, it reflects real-world scenarios where AI-generated content is increasingly dominant.

Each data record contains:

1. Text: Arabic abstract (processed and cleaned)
2. Label: Binary class indicating text origin (Human vs AI)

To ensure data quality, preprocessing techniques were applied, including Arabic normalization, removal of diacritics, tokenization, stopword removal, and stemming. The processed data was then stored in a structured format to support scalable analysis using Apache Spark.

This dataset provides a robust foundation for training and evaluating machine learning models aimed at detecting AI-generated Arabic text.

Methodology

The methodology adopted in this project follows a comprehensive pipeline that integrates big data processing, feature engineering, machine learning modeling, and real-time deployment. The system is designed using Apache Spark to handle large-scale Arabic text efficiently.

1. Data Preprocessing:

The raw Arabic text data was processed through a series of preprocessing steps to improve data consistency and quality. These steps include:

a. Normalization: Standardizing Arabic characters to handle multiple forms of the same letter.
b. Removal of Diacritics: Eliminating diacritical marks (harakat) to simplify text representation.
c. Tokenization: Splitting text into individual words for further analysis.
d. Stopword Removal: Filtering out common Arabic stopwords that do not contribute to classification.
e. Stemming: Reducing words to their root forms to normalize variations in word structure.

The final cleaned text was stored as a new column (abstract_clean) for further processing.

2. Feature Engineering:

To capture meaningful writing patterns, two types of features were used:

a. Traditional Stylometric Features:

Two handcrafted features were extracted using custom Spark UDF functions:
- Number of words with repeated letters: This feature measures repetition patterns in text.
- Total number of sentences: This feature reflects structural differences between texts.

b. Advanced Text Features:

TF-IDF (Term Frequency-Inverse Document Frequency) representation was applied to transform the cleaned text into high-dimensional numeric vectors. This method captures important word-level patterns and their relative importance within the dataset.

3. Data Splitting:

The processed dataset was divided into three subsets:

a. Training set (70%)
b. Validation set (15%)
c. Test set (15%)

This split ensures proper model training and unbiased evaluation.

4. Model Training:

Several machine learning models were implemented using the extracted features:

a. Logistic Regression: A linear model suitable for high-dimensional text classification.
b. Support Vector Machine (SVM): A robust model for separating classes using decision boundaries.
c. XGBoost: A gradient boosting model used for comparison.

Each model was trained using the TF-IDF features, while stylometric features were used for analysis and interpretation.

5. Model Evaluation:

The models were evaluated using standard classification metrics, including:

a. Accuracy
b. F1-score
c. Precision and Recall
d. ROC-AUC

Confusion matrices were also generated to analyze classification performance in more detail.

6. Feature Importance Analysis:

Feature importance was analyzed using the coefficients of the Logistic Regression model. This allowed identification of the most influential features, particularly highlighting the impact of repeated letter patterns compared to sentence count.

7. Real-Time Streaming Implementation:

To simulate deployment, a streaming pipeline was implemented using Spark Structured Streaming. A file-based streaming source was used to mimic real-time data input. Incoming text was processed using the same preprocessing and feature extraction pipeline, followed by real-time classification using the trained model.

8. Scalability Testing:

The system's scalability was evaluated by varying the number of Spark executors and measuring performance metrics such as processing time, throughput, and latency. This analysis demonstrated the system's ability to efficiently process large-scale data in both batch and streaming modes.

Overall, this methodology provides an end-to-end pipeline that combines data preprocessing, feature engineering, machine learning, and real-time analytics within a scalable big data framework.

Results & Analysis

1. The performance of the implemented machine learning models was evaluated using standard classification metrics, including accuracy and F1-score. The evaluation was conducted on a held-out test dataset to ensure an unbiased assessment of model performance.

The experimental results are summarized as follows:

a. Logistic Regression:
Accuracy ≈ 96.5%, F1-score ≈ 0.966

b. Linear SVM:
Accuracy ≈ 96.0%, F1-score ≈ 0.962

c. XGBoost:
Accuracy ≈ 89.5%, F1-score ≈ 0.884

Among all evaluated models, Logistic Regression achieved the highest performance, making it the best-performing model in this study. This result indicates that linear models are highly effective when dealing with high-dimensional sparse representations such as TF-IDF features.

The SVM model also demonstrated strong performance, with results close to Logistic Regression. However, it slightly underperformed in terms of both accuracy and F1-score. Despite this, SVM remains a robust model for text classification tasks and shows strong generalization capability.

In contrast, the XGBoost model showed significantly lower performance compared to the other models. This can be attributed to the high dimensionality and sparsity of TF-IDF features, which are not well-suited for tree-based models. As a result, XGBoost was less effective in capturing the underlying patterns in the text data.

Confusion matrix analysis provides further insight into model behavior. The models showed a strong ability to correctly classify AI-generated text, while a small portion of human-written texts were misclassified. This suggests that AI-generated text tends to exhibit more consistent patterns, making it easier for the models to identify.

Additionally, feature importance analysis revealed that the stylometric feature "number of words with repeated letters" had a stronger impact on classification compared to the "sentence count" feature. The higher absolute coefficient associated with repeated letters indicates that repetition patterns play a key role in distinguishing human-written text from AI-generated content.

Overall, the results demonstrate that combining TF-IDF features with stylometric features significantly improves classification performance. The findings also highlight that linear models are more suitable for handling large-scale text data in this context, while tree-based models may struggle with sparse feature representations.

These results confirm the effectiveness of the proposed approach in detecting AI-generated Arabic text with high accuracy and reliability.

2. Confusion Metrics:
To better understand the classification performance of each model, confusion matrices were generated (see Figures 1-3).

Total abstract for Human 1253: classified correct is 1213, misclassified is 40
Total abstract for AI 5143: classifed correct is 4961, misclassified is 182

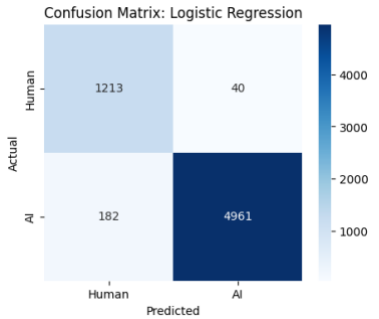

Figure 1: Confusion Matrix for Logistic Regression

Total abstract for Human 1253: classified correct is 1220, misclassified is 33
Total abstract for AI 5143: classifed correct is 4926, misclassified is 217

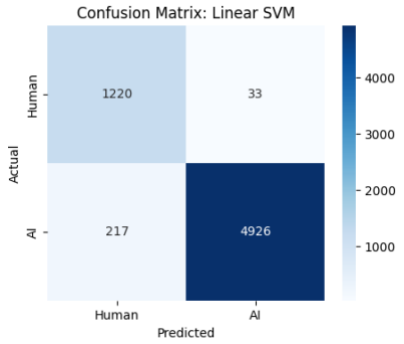

Figure 2: Confusion Matrix for Linear SVM

Total abstract for Human 1253: classified correct is 661, misclassified is 592
Total abstract for AI 5143: classifed correct is 5064, misclassified is 79

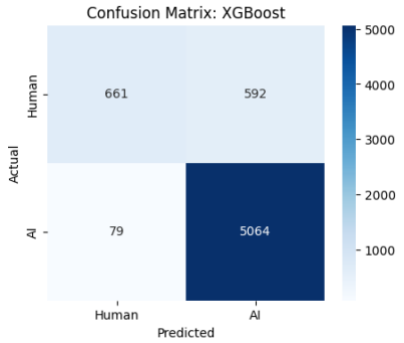

Figure 3: Confusion Matrix for XGBoost

The performance of the evaluated models reveals several important insights regarding their ability to distinguish between human-written and AI-generated Arabic text.

For Logistic Regression, the model achieved strong performance across both classes. Out of 1,253 human-written abstracts, 1,213 were correctly classified while only 40 were misclassified. This reflects a high accuracy in recognizing human-written content. Similarly, for AI-generated abstracts, the model correctly classified 4,961 out of 5,143 samples, with only 182 misclassifications. This demonstrates that Logistic Regression provides a well-balanced performance across both classes, making it a reliable and stable model.

In the case of the Support Vector Machine (SVM), the model showed slightly better performance in detecting human-written texts compared to Logistic Regression, correctly classifying 1,220 out of 1,253 samples with only 33 errors. However, its performance on AI-generated text was slightly lower, with 4,926 correctly classified instances and 217 misclassifications. This indicates that SVM has a stronger ability to recognize human writing patterns but may struggle slightly more with AI-generated text compared to Logistic Regression.

On the other hand, XGBoost exhibited a completely different behavior. While it performed exceptionally well in detecting AI-generated text, correctly classifying 5,064 out of 5,143 samples with only 79 misclassifications, its performance on human-written text was significantly weaker. It correctly classified only 661 out of 1,253 human

samples, while misclassifying 592 instances. This indicates a strong bias toward predicting the AI class, making it unreliable for balanced classification.

Overall, the results show that Logistic Regression is the most balanced model, achieving strong performance across both human and AI classes. SVM also performs well but slightly favors human detection. In contrast, XGBoost demonstrates high accuracy for AI detection but fails to generalize effectively for human-written text, likely due to the nature of high-dimensional sparse features such as TF-IDF.

These findings confirm that linear models are more suitable for this task, while tree-based models may introduce bias when dealing with unevenly distributed text patterns.

3. Batch vs Stream Procesing Trade-offs

Batch processing and stream processing play different roles in this system, each with its own advantages and limitations.

Batch processing works on large amounts of data at once and is mainly used for model training and large-scale analysis. In this project, batch processing achieved stable performance, with execution time remaining nearly constant even when increasing the number of executors. This indicates that batch processing is reliable and efficient for offline processing but does not provide real-time results.

On the other hand, stream processing handles data continuously as it arrives, enabling real-time predictions. The results show that streaming performance improves significantly when increasing the number of executors. Specifically, throughput increases while latency decreases, demonstrating faster response times and better system efficiency.

The trade-off between the two approaches is clear. Batch processing provides higher stability and is more suitable for training and large data analysis, while stream processing offers real-time responsiveness and lower latency, making it ideal for live applications.

Overall, batch processing is best suited for offline tasks, whereas stream processing is essential for real-time systems. Combining both approaches in this project ensures both accuracy and real-time performance.

This demonstrates that integrating both batch and streaming processing creates a balanced system that achieves both accuracy and real-time performance.

Conclusion

This project successfully developed a scalable system for detecting AI-generated Arabic text using big data processing and machine learning techniques. By leveraging Apache Spark, the system was able to efficiently handle large datasets and support both batch and real-time processing.

The experimental results show that Logistic Regression achieved the most balanced and reliable performance, accurately classifying both human-written and AI-generated texts with minimal misclassification. The Support Vector Machine (SVM) also demonstrated strong performance, particularly in detecting human-written content. In contrast, XGBoost showed a strong bias toward identifying AI-generated text, leading to a high number of misclassified human samples, indicating poor generalization on minority class data.

Feature importance analysis revealed that repeated letter patterns are the most influential feature in distinguishing between human and AI-generated text, while sentence count has a relatively smaller impact. This highlights the importance of stylometric patterns in text classification tasks.

The implementation of a real-time streaming pipeline further demonstrated the practical applicability of the system in real-world environments, allowing continuous classification of incoming data. In addition, scalability testing

confirmed that increasing computational resources improves throughput and reduces latency, making the system suitable for large-scale and real-time applications.

Overall, this study confirms that linear models such as Logistic Regression are highly effective for high-dimensional text classification tasks, especially when combined with both TF-IDF and handcrafted features. Future work can explore the use of deep learning models and transformer-based embeddings to further improve classification performance and robustness.